

Security Insider Lab I

Lab 1: Networking Basics, Monitoring Tools, Network Attacks and Defense, and Machine Learning

Group 8

Justlyn Kaniyanthara Johnson 121024
Aayush Kapoor 120957

---

22. Oktober 2026

Contents

1 Introduction **1**
- 1.1 The TCP/IP Stack . . . 1
- 1.2 Core Network Protocols . . . 1
- 1.3 The Border Gateway Protocol . . . 2
- 1.4 ISPs, IXPs and DE-CIX . . . 3

2 Methods **3**
- 2.1 Wireshark . . . 3
- 2.2 Test Setup . . . 3

3 Results **3**
- 3.1 ICMP: Ping . . . 4
- 3.2 DNS Query and Response . . . 4
- 3.3 TCP Three-Way Handshake . . . 4
- 3.4 Encapsulation and HTTP . . . 5
- 3.5 HTTPS and TLS . . . 5

4 Network Attacks and Defense **6**
- 4.1 Five Types of Network Attacks . . . 6
- 4.2 DDoS Attacks . . . 7
  - 4.2.1 Types of DDoS Attacks . . . 7
  - 4.2.2 Reflection and Amplification . . . 8
  - 4.2.3 Scale and Trends . . . 8
- 4.3 Infrastructure Security against DDoS . . . 8
- 4.4 Attack Tools . . . 8
- 4.5 Defense Tools in Detail . . . 9
  - 4.5.1 Cloudflare . . . 9
  - 4.5.2 eBPF and XDP . . . 10

5 Machine Learning for Attack Detection **10**
- 5.1 Why Machine Learning Helps . . . 10
- 5.2 Algorithms for DDoS Detection . . . 11
- 5.3 Evaluation, Data and Challenges . . . 11

6 Discussion **11**
- 6.1 Summary . . . 11
- 6.2 Connecting the Tasks . . . 12
- 6.3 Limitations . . . 12

Bibliography **13**

Eidesstattliche Erklärung **15**

Abstract

This report documents the first lab of the Security Insider Lab. It starts with the networking background the rest of the course builds on: the TCP/IP stack, the core protocols (IP, ICMP, TCP, UDP, DNS, HTTP and TLS), the Border Gateway Protocol, the difference between Internet service providers and Internet exchange points, and DE-CIX in Frankfurt as Germany's largest exchange. Then we put these concepts into practice: using Wireshark 4.2.2 on Ubuntu 24.04 in our own home network, we captured and analyzed a ping, a DNS lookup, a TCP three-way handshake, a plain HTTP request and a TLS-protected connection. The captures matched the textbook in most respects, but a few things surprised us, such as ping running over IPv6, repeated SYN retransmissions during a connection set-up, and a client racing IPv6 against IPv4. Next, the report surveys five network attacks (DDoS, man-in-the-middle, ARP spoofing, DNS spoofing and port scanning) with a closer look at DDoS types, amplification and infrastructure defenses, reviews five attack tools from LOIC to the Aisuru/Kimwolf botnets without running them, and compares two defenses, Cloudflare and eBPF/XDP. The last part explains how machine learning can detect attacks and describes five suitable algorithms: Random Forest, XGBoost, SVM, LSTM and Isolation Forest.

List of Figures

1.1 Encapsulation: each layer adds its own header around the data from the layer above. . 1

3.1 Filter `icmp or icmpv6`. Red: first echo request, blue: its reply, orange: neighbor discovery. 4
3.2 Filter `dns`. Red: A query and response, blue: AAAA query and response. 4
3.3 TCP stream 0. Red: first SYNs (IPv6 and IPv4), orange: retransmissions, blue: SYN-ACK and ACK. 5
3.4 Frame 26 expanded. Left: protocol summary line (green), Ethernet II (blue), IPv6 (orange), TCP (red). Right: TCP flags (red) and options (purple). 5
3.5 The HTTP request (red) and the readable conversation in *Follow TCP Stream*. 6
3.6 TLS Client Hello. Red: the packet, blue: server name, orange: ALPN and supported versions. 6

5.1 Typical detection pipeline. 10

List of Tables

1.1 Division of work in the group. 1
1.2 The four TCP/IP layers. 1
1.3 Recent traffic peaks at DE-CIX Frankfurt as reported in the press. 3

2.1 The Wireshark exercises. 3

4.1 Five network attacks and their main defenses. 7

4.2 The three families of DDoS attacks. 8
4.3 Infrastructure mechanisms against DDoS attacks. 8
4.4 Five attack tools and threats. 9
4.5 Comparison of the two defenses. 10

5.1 Five machine learning algorithms for DDoS detection. 11

1 Introduction

This report covers the first lab of the Security Insider Lab, which has four tasks: networking basics, monitoring tools, network attacks and defense, and machine learning for attack detection. Table 1.1 shows how we divided the work. This chapter covers Task 1 and the background that the later chapters build on. Chapters 2 and 3 turn to Task 2 and show these ideas in real packet captures. Chapter 4 covers attacks and defenses (Task 3), Chapter 5 covers machine learning (Task 4), and Chapter 6 brings the results together.

Task	Topic	Responsible
1	Networking basics	Aayush Kapoor
2	Monitoring with Wireshark	Justlyn K. Johnson
3	Network attacks and defense	Aayush Kapoor
4	Machine learning for detection	Justlyn K. Johnson

Table 1.1: Division of work in the group.

1.1 The TCP/IP Stack

Sending data across the world involves many separate jobs: deciding what to send, getting it there reliably, finding a route and pushing signals through a cable or over Wi-Fi. The TCP/IP stack splits these jobs into four layers (Table 1.2), and each layer only talks to its direct neighbors. The OSI model describes the same idea with seven layers and is mostly used for teaching; TCP/IP is what the Internet actually runs on.

Layer	Question it answers	Examples	Data unit
Application	What is being said?	HTTP, DNS, TLS	message
Transport	Which program gets it?	TCP, UDP (ports)	segment / datagram
Internet	Which machine, which route?	IP, ICMP (IP addresses)	packet
Link	How to cross this one hop?	Ethernet, Wi-Fi (MAC addresses)	frame

Table 1.2: The four TCP/IP layers.

Think of sending a letter: the text is the data, the envelope with the address is the IP header, and the postman's bag is the frame. Every layer wraps the data in its own header, and the link layer also adds a trailer. This is called *encapsulation* (Figure 1.1). The receiver unwraps the layers in reverse order. A router in the middle only unwraps up to the Internet layer, because the destination IP address is all it needs, and then builds a new frame for the next hop. Only the final host passes the data up to the application. Chapter 3 shows this nesting in a captured packet.

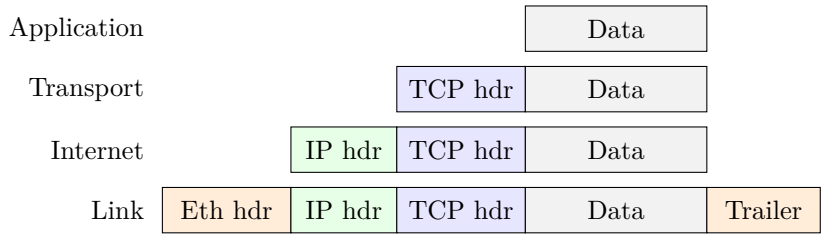


Figure 1.1: Encapsulation: each layer adds its own header around the data from the layer above.

1.2 Core Network Protocols

IP. The Internet Protocol gives every interface an address and moves packets towards their destination on a best-effort basis: no guarantee of delivery, order or that a packet arrives only once [Pos81b]. IPv4

uses 32-bit addresses (about 4.3 billion, which have run out); IPv6 uses 128-bit addresses [DH17]. Nothing in IP checks that a source address is genuine, so it can simply be faked. This *IP spoofing* underlies several of the attacks in Task 3.

ICMP. The Internet Control Message Protocol carries diagnostics and errors, such as "destination unreachable" [Pos81a]. `ping` sends an echo request and waits for the reply. `traceroute` sends packets with an increasing time-to-live (TTL); every router lowers the TTL by one, and the router where it reaches zero answers with an error and thereby reveals itself. ICMPv6 does more and also handles neighbor discovery, the IPv6 replacement for ARP [CDG06; Nar+07].

TCP and UDP. TCP provides a reliable, ordered byte stream: it numbers the data, retransmits what is lost and slows down under congestion [Edd22]. Before any data flows, the two sides open a connection with the three-way handshake (SYN, SYN-ACK, ACK). The handshake does not check anyone's identity; it makes both sides ready and agrees on the starting sequence numbers. UDP is the opposite design: no handshake, no retransmission, almost no overhead [Pos80]. A video call prefers UDP because a late packet is useless, while online banking needs every byte in order and therefore uses TCP.

DNS, HTTP and TLS. The Domain Name System translates names into IP addresses. A client asks a resolver, usually over UDP port 53, separately for IPv4 addresses (record type A) and IPv6 addresses (AAAA) [Moc87]. HTTP is the request/response protocol of the web and runs on TCP [FNR22]. HTTPS is HTTP inside a TLS connection, which adds confidentiality, integrity and server authentication [Res18]. That is why plain HTTP can be read in a capture and HTTPS cannot (Chapter 3).

1.3 The Border Gateway Protocol

The Internet is a collection of tens of thousands of independent networks run by ISPs, universities, companies and cloud providers. Each one is an *autonomous system* (AS) with an AS number. The Border Gateway Protocol (BGP) is how these systems tell each other which IP prefixes they can reach and through which chain of ASes [RLH06]. Each BGP router collects these announcements and, following the policy of its operator, picks a path for every destination. BGP routes *between* networks, while other protocols route inside one.

BGP is a bit like airlines sharing their route maps: a passenger can reach almost any city, but only because every airline publishes where it flies. If an airline stops publishing a route, nobody sends passengers that way. Likewise, a network that stops announcing its prefixes disappears from the map.

BGP also has a trust problem. A neighbor that announces a prefix is believed unless the receiver filters it, so a network can announce address space it does not own (a *hijack*) or pass on routes it should not (a *route leak*). The main countermeasure is RPKI, where the owner of an address block publishes a signed statement of which AS may originate it, so that routers can reject other announcements [LK12].

The Facebook outage of 4 October 2021 shows what happens when BGP goes wrong without any attacker involved. During routine backbone maintenance, a command meant to assess capacity accidentally took down the connections between Facebook's data centers [Jan21]. Facebook's DNS servers were designed to withdraw their own BGP announcements when they cannot reach the data centers, as a safety measure. All of them did exactly that at about 15:40 UTC, and nobody could resolve `facebook.com` any more [Clo21]. The internal repair tools relied on the same network, so engineers had to go to the data centers in person, and the services stayed down for about six hours. BGP and DNS together decide whether a service can be found at all, however many servers are running.

1.4 ISPs, IXPs and DE-CIX

An Internet service provider (ISP), such as Deutsche Telekom or Vodafone, sells Internet access to homes and businesses. An Internet exchange point (IXP) is a different thing: a facility with large switches where many networks, such as ISPs, content providers and clouds, connect to exchange traffic directly. Without an IXP, two networks in the same city might send their traffic through a paid transit provider on a longer path. At an IXP, they connect to the same switch and *peer*, which lowers cost and latency, keeps local traffic local and adds redundancy.

DE-CIX in Frankfurt, Germany's largest IXP, was founded in 1995 by three German ISPs (MAZ, EUnet and XLink), who connected their networks in the back room of an old post office in the Gutleut district to reduce latency and the cost of backhaul [DE-]. As it grew, the switch moved into a data center in the late 1990s, and in 2013 DE-CIX introduced its Apollon platform [Wika]. Today, DE-CIX runs more than 60 exchanges worldwide, and Frankfurt is the largest, with nearly 1,100 connected networks and about 48 exabytes exchanged in 2025 [hes26; Olu26]. Traffic records follow each other quickly (Table 1.3); the 21.47 Tbit/s of October 2026 is more than 160 % above the level of early 2020 [IT 26]. Since these values change almost monthly, the statistics page on `de-cix.net` should be checked for the current number.

Reported	Peak at Frankfurt	Occasion
8 July 2026	19.636 Tbit/s	football match (Argentina v Egypt) [Adv26]
18 September 2026	20.02 Tbit/s	Europa League matches, first time above 20 Tbit/s [Olu26]
14 October 2026	21.47 Tbit/s	new record [IT 26]

Table 1.3: Recent traffic peaks at DE-CIX Frankfurt as reported in the press.

2 Methods

2.1 Wireshark

Wireshark is a free, open-source packet analyzer. It records the frames that pass through a network interface and shows them in three panes: the packet list, the details of the selected packet layer by layer, and the raw bytes. It offers two kinds of filters. A *capture filter* (BPF syntax) is set before recording and discards all other packets for good, whereas a *display filter* only hides packets that are all still stored. We used display filters throughout [Wir].

2.2 Test Setup

All captures were made on one laptop with Ubuntu 24.04.5 LTS and Wireshark 4.2.2, on its Wi-Fi interface (`wlp3s0`) in our own home network (laptop 192.168.178.23, router 192.168.178.1). We only captured our own traffic, on 20 October 2026. For every exercise, we started a fresh capture, generated the traffic, stopped the capture and applied a display filter. Table 2.1 lists the exercises.

Exercise	Traffic generated	Display filter
1 ICMP	`ping -c 4 google.com`	`icmp or icmpv6`
2 DNS	Prime Video in the browser, `nslookup primevideo.com`	`dns`
3 TCP and HTTP	`curl -I http://neverssl.com`	`tcp.flags.syn == 1 or tcp.flags.ack == 1 and tcp.stream eq 0`, then `tcp.stream eq 0` and *Follow TCP Stream*
4 TLS	`curl https://google.com`	`tls`

Table 2.1: The Wireshark exercises.

3 Results

3.1 ICMP: Ping

The obvious filter for `ping -c 4 google.com` is `icmp`, but it would show nothing here: `google.com` was resolved to an IPv6 address, so the ping used ICMPv6. With `icmp or icmpv6`, the four echo requests and replies appear (Figure 3.1). They share the identifier `0x8cf7` and are matched by sequence number. The requests leave with hop limit 64 (the IPv6 name for TTL) and the replies arrive with 111. The round-trip times calculated from the time stamps are between 9.5 and 12.0 ms, about 11.2 ms on average. The Ethernet destination of each request is the MAC address of our router, not that of Google: the link layer only moves the frame to the next hop. Below the pings, ICMPv6 Neighbor Solicitation and Advertisement messages (orange) are the IPv6 counterpart of ARP.

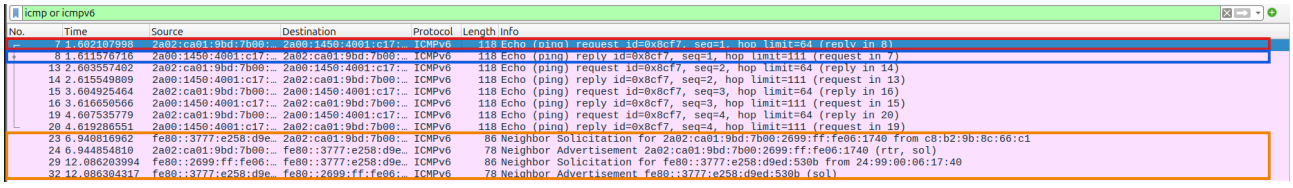


Figure 3.1: Filter `icmp or icmpv6`. Red: first echo request, blue: its reply, orange: neighbor discovery.

3.2 DNS Query and Response

The filter `dns` leaves 78 of the 3 921 captured packets (Figure 3.2). The laptop sends its queries to the home router (192.168.178.1), which forwards them upstream. Each query is a UDP datagram from a random source port (here 50773) to port 53, and the answer returns from port 53 and carries the same transaction ID (`0x9cc3` and `0xa362`). `nslookup` asks for both record types. The A query for `primevideo.com` was answered after 17.9 ms with four IPv4 addresses (18.66.102.x). The AAAA query was answered after 11.2 ms, but without an address, only with a start-of-authority (SOA) record: the name simply had no IPv6 address. Oddly, the queries still travel over IPv4 even though the laptop has working IPv6.

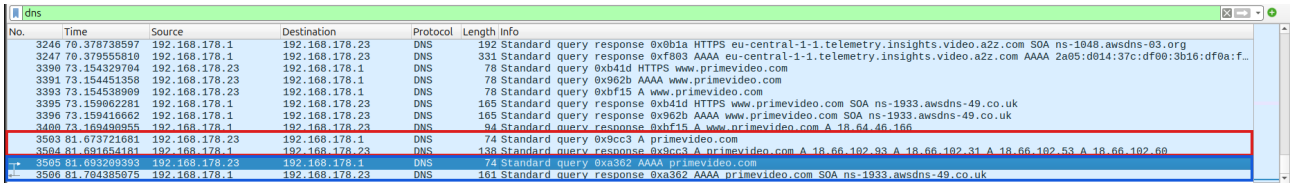


Figure 3.2: Filter `dns`. Red: A query and response, blue: AAAA query and response.

3.3 TCP Three-Way Handshake

For the handshake, we looked at the connection of `curl` to `neverssl.com` (TCP stream 0), a deliberately unencrypted site. The filter `tcp.flags.syn == 1 or tcp.flags.ack == 1 and tcp.stream eq 0` contains a trap: `and` binds more strongly than `or`, so it shows *every* SYN in the capture plus the ACKs of stream 0. That is why IPv4 attempts show up in Figure 3.3; adding parentheses gives a cleaner view. The handshake consists of frames 26 (SYN), 35 (SYN-ACK) and 36 (ACK). Wireshark shows relative sequence numbers: `Seq=0`, then `Seq=0, Ack=1`, then `Seq=1, Ack=1`. The acknowledgment is one higher because a SYN consumes one sequence number. The SYN also carries options such as a maximum segment size of 1432 bytes and a window scale of 10 (a multiplier of 1024).

The capture also shows something a textbook diagram leaves out. The IPv6 SYN was sent at 9.1 s, but the SYN-ACK arrived only at 12.4 s, after three retransmissions of the SYN at intervals of about one second. Later, the HTTP request itself was retransmitted four times with roughly doubling intervals

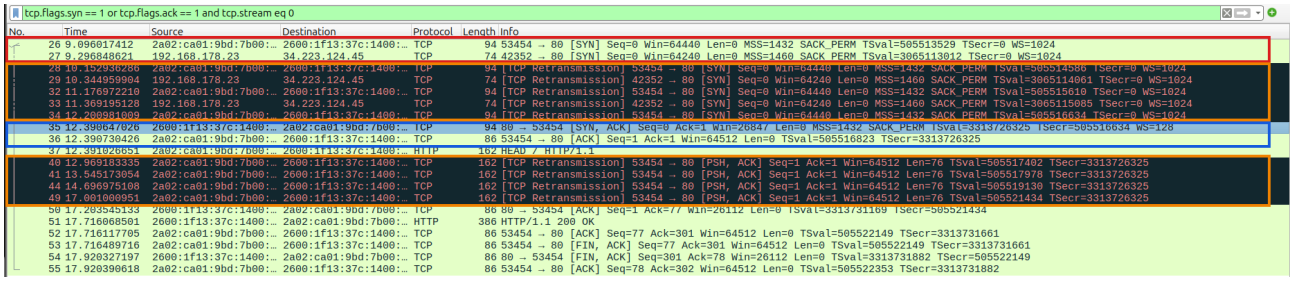


Figure 3.3: TCP stream 0. Red: first SYNs (IPv6 and IPv4), orange: retransmissions, blue: SYN-ACK and ACK.

(0.58 up to 2.3 s) before the server acknowledged it. We cannot tell from the capture whether Wi-Fi, the access network or the server was responsible, but it shows why TCP numbers its data and retransmits. We also noticed a pair of IPv6 and IPv4 SYNs only about 200 ms apart (frames 26 and 27). `curl` had started an IPv6 connection and a fallback over IPv4 (Happy Eyeballs [SP17]), and only the IPv6 attempt completed.

3.4 Encapsulation and HTTP

Figure 3.4 expands all layers of the first SYN (frame 26, 94 bytes). The line `Protocols in frame: eth:ethertype:ipv6:tcp` summarizes the nesting from Figure 1.1. The Ethernet header holds the MAC addresses (our adapter and the router) and the type `0x86dd` for IPv6. The IPv6 header holds the 128-bit addresses and the hop limit. The TCP header holds the ports (53454 to 80) and the flags, here only SYN. The sizes add up: $14 + 40 + 40 = 94$ bytes.

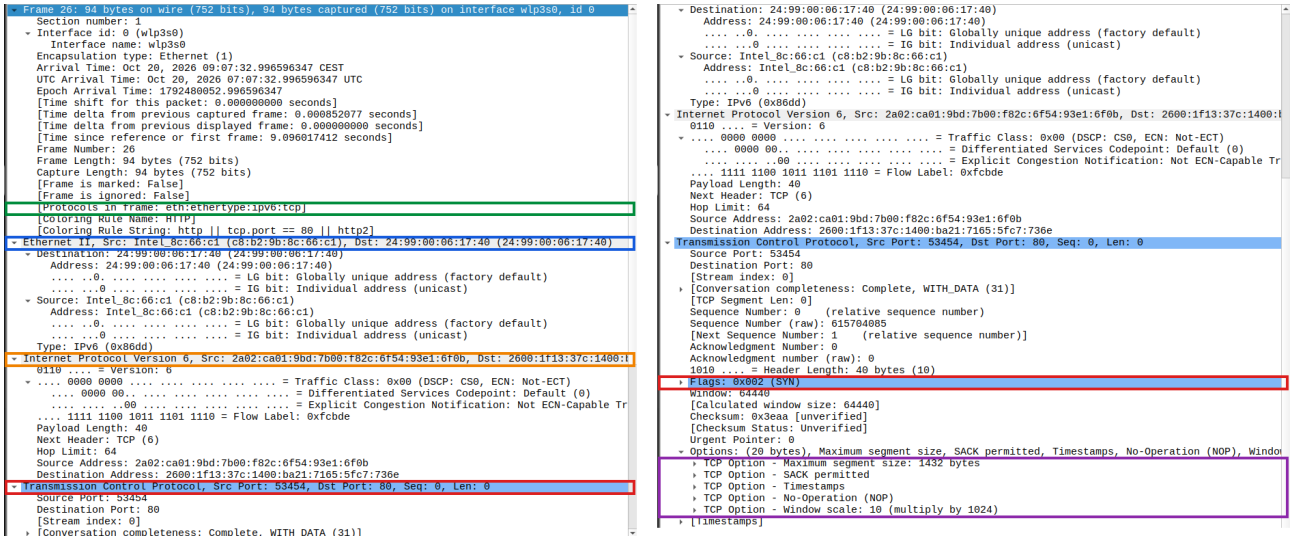


Figure 3.4: Frame 26 expanded. Left: protocol summary line (green), Ethernet II (blue), IPv6 (orange), TCP (red). Right: TCP flags (red) and options (purple).

The same stream carries the application layer. *Follow TCP Stream* (Figure 3.5) shows the whole conversation in readable text: a `HEAD / HTTP/1.1` request with `Host: neverssl.com` and `User-Agent: curl/8.5.0`, and a `200 OK` response from an Apache server. Anyone on the path can read this, including the server software and the client program, which is why HTTPS is the default today.

3.5 HTTPS and TLS

For HTTPS, we ran `curl https://google.com` and filtered on `tls`. The connection starts with a Client Hello (frame 20, Figure 3.6), which is still readable: it reveals the server name in the `server_name`



Figure 3.5: The HTTP request (red) and the readable conversation in *Follow TCP Stream*.

extension (SNI: `google.com`), the offered protocols (`h2` and `http/1.1`) and the supported TLS versions (1.3 and 1.2). The Server Hello followed 18.6 ms later. From there on, everything is encrypted. In TLS 1.3, even the certificate and the rest of the handshake are encrypted, so Wireshark labels them "Application Data", and the content cannot be recovered without the session keys. The metadata (addresses, ports, sizes, timing and the SNI) remains visible.

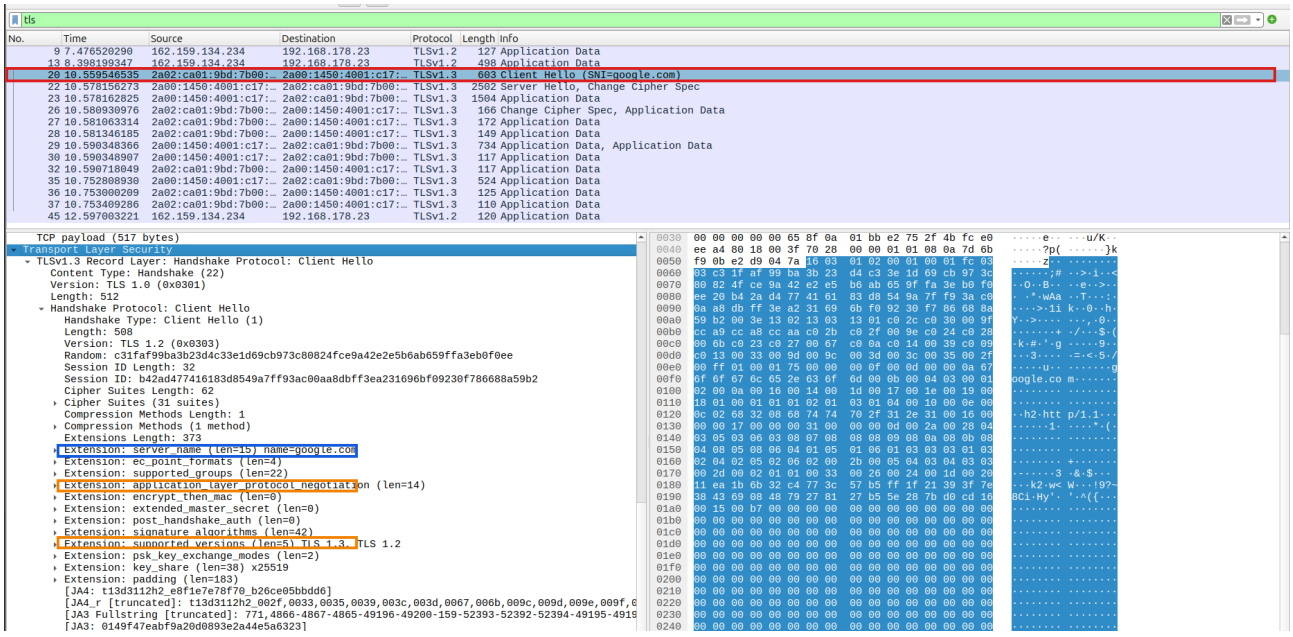


Figure 3.6: TLS Client Hello. Red: the packet, blue: server name, orange: ALPN and supported versions.

4 Network Attacks and Defense

This chapter covers Task 3 and is based on literature and vendor reports only. We did not run any attack tool; Section 4.4 only describes them. Using them against systems one does not own can be a crime in Germany (computer sabotage, § 303b StGB).

4.1 Five Types of Network Attacks

We picked five attacks that hit different layers of the stack (Table 4.1).

Man-in-the-middle (MitM). The attacker relays traffic between two parties who think they talk directly to each other, for instance through a rogue Wi-Fi access point. Against plain HTTP, the

Attack	Idea (and an everyday analogy)	Main defenses
DDoS	Many machines flood a target so that real users cannot get through (a crowd blocking a shop door).	rate limiting, scrubbing, anycast and CDN, anti-spoofing
Man-in-the-middle	An attacker sits unnoticed between two parties and reads or changes their traffic (a postman who opens letters).	TLS with certificate checks, VPN, avoiding untrusted Wi-Fi
ARP spoofing	On a local network, fake ARP replies make victims send frames to the attacker's MAC address instead of the router's (swapping the name sign on a door).	dynamic ARP inspection, static entries, port security
DNS spoofing	Forged DNS answers make a name point to the attacker's IP address (a wrong entry in the phone book).	DNSSEC [Are+05], randomized source ports, trusted resolvers
Port scanning	Probing a host for open ports and services, usually as reconnaissance (trying all doors of a building).	firewall, closing unused services, intrusion detection

Table 4.1: Five network attacks and their main defenses.

attacker can read and change everything, as the capture in Section 3.4 shows. Against HTTPS, the attacker would also need a certificate that the browser accepts, which is exactly the protection TLS provides [Res18].

ARP spoofing. ARP maps IPv4 addresses to MAC addresses on a local network [Plu82] and has no authentication, so any host can claim "the router's IP address is at my MAC address". Victims then send their traffic to the attacker, which is a MitM position. Switches can counter this with dynamic ARP inspection. In IPv6, the neighbor discovery messages from Figure 3.1 play the same role and can be abused similarly.

DNS spoofing. If a forged answer is accepted by a resolver, every user of that resolver is sent to the wrong address. Plain DNS runs over unauthenticated UDP (Section 3.2), so protection relies on unpredictable transaction IDs and source ports and, more fundamentally, on signatures with DNSSEC [Are+05].

Port scanning. A scan probes many ports and records which ones answer, which reveals running services and often their versions. The best-known scanner is Nmap, which is legitimate on one's own systems [Lyo]. Scans do no damage but usually come before a real attack, so firewalls and intrusion detection systems watch for them.

4.2 DDoS Attacks

A denial-of-service (DoS) attack makes a service unavailable. In a *distributed* DoS (DDoS), the traffic comes from many machines at the same time, usually a *botnet*: thousands or millions of compromised devices such as cameras, routers and TV boxes, which an attacker controls through a command-and-control (C2) server. The owners typically do not notice. Because the traffic comes from everywhere, simply blocking one source does not help.

4.2.1 Types of DDoS Attacks

DDoS attacks fall into three families, depending on which resource they exhaust (Table 4.2).
The SYN flood connects directly to our captures. A server that receives a SYN reserves memory for a half-open connection and answers with SYN-ACK. If SYNs arrive in huge numbers with spoofed sources, the final ACK never comes, the table of half-open connections fills up and real users are refused.

Family	Target resource	Examples	Measured in
Volumetric	bandwidth	UDP floods, DNS and NTP amplification	bit/s
Protocol	state and capacity of servers and network devices	SYN flood, fragmentation attacks, ping of death	packets/s
Application layer	the application itself	HTTP GET/POST floods, Slowloris	requests/s

Table 4.2: The three families of DDoS attacks.

The connection is never established; the damage is resource exhaustion. The standard countermeasure is SYN cookies, where the server keeps no state until the handshake is complete [Edd07].

4.2.2 Reflection and Amplification

In a reflection attack, a third-party server delivers the traffic: the attacker sends a request with the *victim's* address as spoofed source, and the reply goes to the victim. In an amplification attack, the reply is much larger than the request. This works well over UDP, which has no handshake that would expose the spoofed address (Section 1.2). CISA lists bandwidth amplification factors of 28 to 54 for DNS, about 557 for NTP and 10 000 to 51 000 for memcached [CIS], so a request of a few dozen bytes can become megabytes aimed at the victim.

4.2.3 Scale and Trends

DDoS attacks have grown quickly. Cloudflare reports about 47.1 million attacks in 2025, an increase of 121 % over 2024, with telecommunications providers and ISPs as the main targets [The26; Fas26; Cyb26]. A single attack in late 2025 reached 31.4 Tbit/s but lasted only 35 seconds and was mitigated automatically [The26]. Such short, extreme bursts are hard to handle by hand, which is why detection is automated (Chapter 5).

4.3 Infrastructure Security against DDoS

No single measure stops all DDoS attacks, so operators combine several (Table 4.3).

Mechanism	What it does
Rate limiting	Caps requests or packets per source or per second.
Firewalls and ACLs	Drop traffic that does not match the allowed rules.
Load balancing	Spreads the load over many servers so that none of them collapses.
Anycast and CDN	The same IP address is announced from many locations by BGP, so attack traffic is split up and absorbed close to where it enters the network.
Scrubbing centers	Traffic is diverted to a facility that removes the attack traffic and forwards the clean part.
Blackholing (RTBH)	All traffic to the attacked IP address is dropped at the network edge or at an IXP. The network survives, but the victim address is offline.
Anti-spoofing (BCP 38)	Networks drop packets whose source address cannot be genuine, which stops spoofed floods at their origin [FS00].
SYN cookies	The server stores no state until the TCP handshake is complete.
Over-provisioning	Having more bandwidth and capacity than an attacker can send.

Table 4.3: Infrastructure mechanisms against DDoS attacks.

Several of these link back to Task 1: anycast is a BGP technique, blackholing is often done at an IXP such as DE-CIX, and BCP 38 only works if every network along the path takes part.

4.4 Attack Tools

Table 4.4 lists five attack tools and threats from older to newer. The older ones show the principles; the last two explain today's threat level.

Tool	What it is	Why it matters
LOIC (Low Orbit Ion Cannon)	Open-source flooding tool with a GUI that sends TCP, UDP or HTTP floods [Wikb]. Known from Anonymous campaigns around 2010.	Very easy to use, but it does not hide the user's address, so participants could be traced.
Slowloris	Opens many connections to a web server and sends HTTP headers extremely slowly, so that the server's connections stay occupied [Clo].	Needs very little bandwidth; a good example of an application-layer attack.
hping3	Command-line tool that crafts custom TCP, UDP and ICMP packets, including SYN floods and spoofed addresses [San].	Also a legitimate test tool for firewalls, so it is dual-use.
Mirai	IoT botnet malware from 2016 that logs into devices with default passwords and turns them into bots [Ant+17]. Its source code was published, so many variants exist.	Its attack on the DNS provider Dyn in 2016 disrupted many well-known sites; it still shapes botnets today.
Aisuru / Kimwolf	Recent botnets built from compromised IoT devices and, in the case of Kimwolf, mostly off-brand Android TV boxes, with an estimated one to several million devices [The26; Cyb26].	Responsible for the record attacks of 2025 (29.7 and 31.4 Tbit/s [Clo25; The26]); the most relevant current example.

Table 4.4: Five attack tools and threats.

Looking from LOIC to Aisuru shows a clear shift. LOIC needed volunteers to run it on their own computers, which limited its size and exposed them. Mirai and its successors replaced volunteers with hijacked devices, so attackers need nobody's cooperation and are much harder to trace. Attack size is now limited by the number of insecure devices on the Internet, not by the attacker's own resources.

4.5 Defense Tools in Detail

We selected two defenses that work at very different places: Cloudflare, which defends at the network edge in many locations, and eBPF/XDP, which defends on a single Linux host.

4.5.1 Cloudflare

Cloudflare is a content delivery network that sits between its customers' servers and the Internet. Traffic for a protected site reaches the nearest Cloudflare data center through BGP anycast, so an attack is split over many locations instead of hitting one [Yoa]. In each data center, XDP programs called L4Drop first apply the rules of known attacks and send traffic samples to a detection service named `dosd`. When `dosd` recognizes an attack, it creates a mitigation rule, which, according to Cloudflare, takes seconds and not minutes [IT]. A second system, Gatebot, watches for attacks across the whole network, and HTTP attacks are handled in the web proxy [Yoa]. This is how Cloudflare mitigated the 31.4 Tbit/s attack of 2025 without human intervention [The26].

The limits are mostly organizational. The customer depends on a third party and its prices, and since traffic passes through the provider, it has to trust it with the data (for HTTPS, the provider typically terminates TLS). Attackers may also bypass the protection by hitting the origin server's real address, if they know it.

4.5.2 eBPF and XDP

eBPF lets small, verified programs run inside the Linux kernel without changing the kernel. The eXpress Data Path (XDP) is a hook for such programs at the earliest possible point, in the network driver, before the kernel has allocated its usual packet structures [Høi+18]. A program can inspect each packet, look up its source in a table (an eBPF map) and pass or drop it. Dropping early saves most of the work of the network stack, so one server can discard packets at very high rates. Think of a bouncer who turns visitors away at the door, before they reach the receptionist. Cloudflare uses XDP in exactly this way for L4Drop [Yoa].

The limits differ from Cloudflare's. XDP needs Linux and programming expertise, and the operator writes and maintains the rules. The bigger limit is that it protects the host and not the link: if an attack fills the incoming link, packets are lost before they reach the machine. This is why the two defenses complement each other (Table 4.5). We will work with XDP in Lab 4.

	Cloudflare	eBPF / XDP
Where	in the provider's network, in many locations	on one Linux host, in the network driver
Strength	absorbs very large volumes; automatic	very fast packet dropping; full control, no third party
Weakness	dependency, cost, trust in the provider	cannot protect a saturated link; needs expertise
Best against	volumetric and application-layer floods	protocol floods that reach the host (e.g. SYN, UDP)

Table 4.5: Comparison of the two defenses.

5 Machine Learning for Attack Detection

This chapter covers Task 4. The idea behind it is simple: attack traffic looks statistically different from normal traffic. A machine learning model learns these differences from examples and then flags new traffic that looks like an attack, quickly and automatically.

5.1 Why Machine Learning Helps

A fixed rule such as "block everything above 100 requests per second" is easy to evade and also hits legitimate users during a busy moment, because a flash crowd looks like an attack. A model can combine many signals and adapt when it is retrained. This matters because attacks are increasingly short and fast (Section 4.2), so detection must not wait for a human.

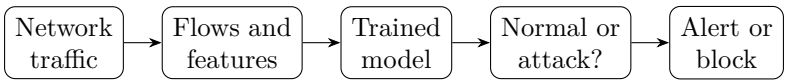


Figure 5.1: Typical detection pipeline.

Raw packets are first grouped into *flows* (all packets of one conversation) and summarized as numbers called *features* (Figure 5.1). Typical features are packets and bytes per second, the mean and variation of packet sizes, the number of packets with SYN or ACK flags, the flow duration, the number of distinct source addresses and the entropy (randomness) of source addresses and ports. In a DDoS attack, the last two change visibly, because many sources send similar packets.

Models are trained in one of two ways. *Supervised* learning uses labeled examples ("normal" or "attack") and reaches high accuracy on attack types it has seen, but it knows nothing about new ones. *Unsupervised* learning, also called anomaly detection, learns what normal traffic looks like and flags anything unusual. It can detect unseen attacks but produces more false alarms.

5.2 Algorithms for DDoS Detection

Table 5.1 lists five algorithms that are used for DDoS detection: four supervised ones and one for anomaly detection.

Algorithm	How it works	Why it fits	Limitation
Random Forest [Bre01]	Many decision trees, each trained on a random part of the data, vote on the class.	Accurate, robust, handles many features; very common in DDoS research.	Larger models are slower to evaluate; needs labels.
XGBoost [CG16]	Builds trees one after another, each correcting the errors of the previous ones.	Often reaches the best accuracy on tabular flow features.	Needs careful tuning; needs labels.
Support Vector Machine [CV95]	Finds the boundary that separates normal from attack with the widest margin.	Works well with many features and moderate data sizes.	Training becomes slow for very large datasets.
LSTM [HS97]	A neural network with memory that reads a sequence of flows or packets in order.	Captures how traffic changes over time.	Needs more data and computing power; harder to explain.
Isolation Forest [LTZ08]	Random trees try to isolate each sample; unusual samples are isolated with fewer splits.	Needs no labels, so it can flag unseen attacks.	More false positives; the threshold for "unusual" must be chosen.

Table 5.1: Five machine learning algorithms for DDoS detection.

5.3 Evaluation, Data and Challenges

Accuracy alone is a poor measure for a detector. Normal traffic is much more common than attack traffic, so a model that always answers "normal" can look very accurate and still detect nothing. More useful are *precision* (of the alarms raised, how many were real attacks), *recall* (of the real attacks, how many were found), the F1 score (a combination of both) and the false positive rate (legitimate users wrongly blocked).

A commonly used public dataset is CIC-DDoS2019 of the Canadian Institute for Cybersecurity, which contains labeled flows for many DDoS types, including reflection attacks over DNS, NTP and UDP [Sha+19]. We expect to use it in Labs 2 and 3. Four challenges should be kept in mind:

- **Old data.** Attacks change, so a model trained on an old dataset can fail on new traffic (concept drift).
- **False positives.** Blocking real users is costly, so a detector must be tuned carefully.
- **Speed.** Detection must work in real time at very high packet rates.
- **Evasion.** Attackers can adapt their traffic so that it looks normal.

6 Discussion

6.1 Summary

The captures in Chapter 3 confirmed the concepts of Task 1 and added details that textbook diagrams leave out. Encapsulation was visible in a single frame (14 bytes of Ethernet, 40 of IPv6 and 40 of TCP), the handshake showed sequence numbers rising by one for the SYN, and the HTTP exchange was fully readable while the TLS exchange was not. The captures also held surprises: ping over IPv6, DNS over IPv4 although IPv6 worked, a connection that needed three SYN retransmissions, and a

client racing IPv6 against IPv4. Real traffic is rarely as tidy as the schematic version.

6.2 Connecting the Tasks

Some observations from Task 2 point directly to the attacks of Task 3:

- The server commits resources when it answers a SYN (Section 3.3). A SYN flood abuses exactly this step, and SYN cookies defend against it.
- DNS runs over UDP without a handshake (Section 3.2), which is what makes spoofing and amplification possible (Section 4.2).
- The plain HTTP request exposed the server software and the client program to anyone on the path (Section 3.4). TLS hides the content but not metadata such as the server name (Section 3.5).
- The Facebook outage (Section 1.3) and anycast-based DDoS protection (Section 4.3) both rely on BGP, once as a point of failure and once as a tool.

The two defenses of Section 4.5 complement each other: a provider like Cloudflare absorbs volume far from the victim, while XDP drops unwanted packets cheaply at the host. Machine learning (Chapter 5) can supply the detection part that tells such filters *what* to drop, which is the direction of Labs 2 to 4.

6.3 Limitations

Our measurements come from one laptop on one home network on one day, with captures of a few dozen to a few thousand packets. They illustrate protocol behavior but cannot support statistical claims, and we could not determine the cause of the SYN retransmissions from the capture alone. The attack and defense chapters rely on published reports, whose numbers change quickly; we checked them on 20 October 2026. The attack tools were described from the literature and none was run.

Bibliography

[Adv26] Advanced Television. 2026. URL: https://www.advanced-television.com/2026/07/08/de_cix-argentina-vs-egypt-breaks-de-cix-global-traffic-record/ (visited on 10/20/2026) (cit. on p. 3).

[Ant+17] Manos Antonakakis et al. "Understanding the Mirai Botnet." In: *26th USENIX Security Symposium*. 2017 (cit. on p. 9).

[Are+05] Roy Arends et al. *DNS Security Introduction and Requirements*. RFC 4033. IETF, 2005. URL: https://www.rfc-editor.org/rfc/rfc4033 (cit. on p. 7).

[Bre01] Leo Breiman. "Random Forests." In: *Machine Learning* 45.1 (2001), pp. 5–32 (cit. on p. 11).

[CG16] Tianqi Chen and Carlos Guestrin. "XGBoost: A Scalable Tree Boosting System." In: *Proc. ACM SIGKDD*. 2016 (cit. on p. 11).

[CIS] CISA. *Alert TA14-017A: UDP-Based Amplification Attacks*. URL: https://www.cisa.gov/ncas/alerts/TA14-017A (visited on 10/20/2026) (cit. on p. 8).

[DE-] DE-CIX. *30 years of DE-CIX: How Frankfurt became the heart of the European Internet*. URL: https://www.de-cix.net/en/30years/frankfurt-heart-of-european-internet (visited on 10/20/2026) (cit. on p. 3).

[Clo21] Cloudflare. *Understanding how Facebook disappeared from the Internet*. 2021. URL: https://blog.cloudflare.com/october-2021-facebook-outage/ (visited on 10/20/2026) (cit. on p. 2).

[Clo25] Cloudflare. *DDoS threat report for 2025 Q3*. 2025. URL: https://blog.cloudflare.com/ddos-threat-report-2025-q3/ (visited on 10/20/2026) (cit. on p. 9).

[Clo] Cloudflare. *Slowloris DDoS attack*. Cloudflare Learning Center. URL: https://www.cloudflare.com/learning/ddos/ddos-attack-tools/slowloris/ (visited on 10/20/2026) (cit. on p. 9).

[CDG06] Alex Conta, Stephen Deering, and Mukesh Gupta. *Internet Control Message Protocol (ICMPv6) for the Internet Protocol Version 6 (IPv6) Specification*. RFC 4443. IETF, 2006. URL: https://www.rfc-editor.org/rfc/rfc4443 (cit. on p. 2).

[CV95] Corinna Cortes and Vladimir Vapnik. "Support-Vector Networks." In: *Machine Learning* 20.3 (1995), pp. 273–297 (cit. on p. 11).

[Cyb26] CyberInsider. 2026. URL: https://cyberinsider.com/cloudflare-mitigates-record-breaking-31-4-tbps-ddos-attack-in-q4-2025/ (visited on 10/20/2026) (cit. on pp. 8, 9).

[DH17] Stephen Deering and Robert Hinden. *Internet Protocol, Version 6 (IPv6) Specification*. RFC 8200. IETF, 2017. URL: https://www.rfc-editor.org/rfc/rfc8200 (cit. on p. 2).

[Edd07] Wesley Eddy. *TCP SYN Flooding Attacks and Common Mitigations*. RFC 4987. IETF, 2007. URL: https://www.rfc-editor.org/rfc/rfc4987 (cit. on p. 8).

[Edd22] Wesley Eddy. *Transmission Control Protocol (TCP)*. RFC 9293. IETF, 2022. URL: https://www.rfc-editor.org/rfc/rfc9293 (cit. on p. 2).

[Fas26] FastNetMon. 2026. URL: https://fastnetmon.com/2026/02/01/aisuru-botnet-sets-a-new-ddos-record-at-31-4-tbps/ (visited on 10/20/2026) (cit. on p. 8).

[FS00] Paul Ferguson and Daniel Senie. *Network Ingress Filtering: Defeating Denial of Service Attacks which employ IP Source Address Spoofing*. RFC 2827. IETF, 2000. URL: https://www.rfc-editor.org/rfc/rfc2827 (cit. on p. 8).

[FNR22] Roy Fielding, Mark Nottingham, and Julian Reschke. *HTTP Semantics*. RFC 9110. IETF, 2022. URL: https://www.rfc-editor.org/rfc/rfc9110 (cit. on p. 2).

[hes26] hessenschau.de. *Frankfurts Internetknoten knackt Rekord dank Champions League*. 2026. URL: https://www.hessenschau.de/wirtschaft/frankfurts-internetknoten-knackt-rekord-dank-champions-league-v1,internetknotenpunkt-datenverkehr-steigt-100.html (visited on 10/20/2026) (cit. on p. 3).

[HS97] Sepp Hochreiter and Jürgen Schmidhuber. "Long Short-Term Memory." In: *Neural Computation* 9.8 (1997), pp. 1735–1780 (cit. on p. 11).

[Høi+18] Toke Høiland-Jørgensen et al. "The eXpress Data Path: Fast Programmable Packet Processing in the Operating System Kernel." In: *Proc. ACM CoNEXT*. 2018 (cit. on p. 10).

[IT 26] IT Brief UK. 2026. URL: https://itbrief.co.uk/story/frankfurt-internet-exchange-hits-record-21-47-terabits (visited on 10/20/2026) (cit. on p. 3).

[IT] IT Brief UK. *Cloudflare uses eBPF to boost edge security & routing*. URL: https://itbrief.co.uk/story/cloudflare-uses-ebpf-to-boost-edge-security-routing (visited on 10/20/2026) (cit. on p. 9).

[Jan21] Santosh Janardhan. *More details about the October 4 outage*. Engineering at Meta. 2021. URL: https://engineering.fb.com/2021/10/05/networking-traffic/outage-details/ (visited on 10/20/2026) (cit. on p. 2).

[LK12] Matt Lepinski and Stephen Kent. *An Infrastructure to Support Secure Internet Routing*. RFC 6480. IETF, 2012. URL: https://www.rfc-editor.org/rfc/rfc6480 (cit. on p. 2).

[LTZ08] Fei Tony Liu, Kai Ming Ting, and Zhi-Hua Zhou. "Isolation Forest." In: *Proc. IEEE ICDM*. 2008 (cit. on p. 11).

[Lyo] Gordon Lyon. *Nmap Network Scanning*. URL: https://nmap.org/book/ (visited on 10/20/2026) (cit. on p. 7).

[Moc87] Paul Mockapetris. *Domain Names – Implementation and Specification*. RFC 1035. IETF, 1987. URL: https://www.rfc-editor.org/rfc/rfc1035 (cit. on p. 2).

[Nar+07] Thomas Narten et al. *Neighbor Discovery for IP version 6 (IPv6)*. RFC 4861. IETF, 2007. URL: https://www.rfc-editor.org/rfc/rfc4861 (cit. on p. 2).

[Olu26] Peter Oluka. 2026. URL: https://techeconomy.ng/europa-league-streaming-breaks-20-terabit-internet-record (visited on 10/20/2026) (cit. on p. 3).

[Plu82] David C. Plummer. *An Ethernet Address Resolution Protocol*. RFC 826. IETF, 1982. URL: https://www.rfc-editor.org/rfc/rfc826 (cit. on p. 7).

[Pos80] Jon Postel. *User Datagram Protocol*. RFC 768. IETF, 1980. URL: https://www.rfc-editor.org/rfc/rfc768 (cit. on p. 2).

[Pos81a] Jon Postel. *Internet Control Message Protocol*. RFC 792. IETF, 1981. URL: https://www.rfc-editor.org/rfc/rfc792 (cit. on p. 2).

[Pos81b] Jon Postel. *Internet Protocol*. RFC 791. IETF, 1981. URL: https://www.rfc-editor.org/rfc/rfc791 (cit. on p. 1).

[RLH06] Yakov Rekhter, Tony Li, and Susan Hares. *A Border Gateway Protocol 4 (BGP-4)*. RFC 4271. IETF, 2006. URL: https://www.rfc-editor.org/rfc/rfc4271 (cit. on p. 2).

[Res18] Eric Rescorla. *The Transport Layer Security (TLS) Protocol Version 1.3*. RFC 8446. IETF, 2018. URL: https://www.rfc-editor.org/rfc/rfc8446 (cit. on pp. 2, 7).

[San] Salvatore Sanfilippo. *hping3*. URL: https://www.hping.org/ (visited on 10/20/2026) (cit. on p. 9).

[SP17] David Schinazi and Tommy Pauly. *Happy Eyeballs Version 2: Better Connectivity Using Concurrency*. RFC 8305. IETF, 2017. URL: https://www.rfc-editor.org/rfc/rfc8305 (cit. on p. 5).

[Sha+19] Iman Sharafaldin et al. "Developing Realistic Distributed Denial of Service (DDoS) Attack Dataset and Taxonomy." In: *Proc. IEEE International Carnahan Conference on Security Technology*. 2019 (cit. on p. 11).

[The26] The Hacker News. *AISURU/Kimwolf Botnet Launches Record-Setting 31.4 Tbps DDoS Attack*. 2026. URL: https://thehackernews.com/2026/02/aisurukimwolf-botnet-launches-record.html (visited on 10/20/2026) (cit. on pp. 8, 9).

[Wika] Wikipedia. URL: https://en.wikipedia.org/wiki/DE-CIX (visited on 10/20/2026) (cit. on p. 3).

[Wikb] Wikipedia. *Low Orbit Ion Cannon*. URL: https://en.wikipedia.org/wiki/Low_Orbit_Ion_Cannon (visited on 10/20/2026) (cit. on p. 9).

[Wir] Wireshark Foundation. *Wireshark User's Guide*. URL: https://www.wireshark.org/docs/wsug_html_chunked/ (visited on 10/20/2026) (cit. on p. 3).

[Yoa] Omer Yoachimik. *A deep-dive into Cloudflare's autonomous edge DDoS protection*. Cloudflare Blog. URL: https://blog.cloudflare.com/deep-dive-cloudflare-autonomous-edge-ddos-protection/ (visited on 10/20/2026) (cit. on pp. 9, 10).

Eidesstattliche Erklärung

Hiermit versichere ich, dass ich diese MasterLab selbstständig und ohne Benutzung anderer als der angegebenen Quellen und Hilfsmittel angefertigt habe und alle Ausführungen, die wörtlich oder sinngemäß übernommen wurden, als solche gekennzeichnet sind, sowie, dass ich die MasterLab in gleicher oder ähnlicher Form noch keiner anderen Prüfungsbehörde vorgelegt habe.

Passau, 22. Oktober 2026

Justlyn Kaniyanthara Johnson

Aayush Kapoor